

Arquivos Binário

Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Arquivos Binário em C/C++

- Arquivos em C/C++ não podem ser associados a tipo primitivo ou registro
 - Armazenam sequência de caracteres ou de *bytes*
- Em vários momentos, é útil:
 - Ler de um arquivo e gravar diretamente em variável **int** ou **float**, ou, ainda em uma variável **struct**
 - Gravar o conteúdo de uma variável em um arquivo
- Isto é **arquivo binário**
 - Uma operação de leitura ou escrita deverá ser informar o número de *bytes* que serão lidos ou gravados

fwrite

A função **`fwrite()`** pode escrever qualquer tipo de dado, e não apenas caracteres ou cadeia de caracteres. A sintaxe é:

```
int fwrite(void *mem, size_t qtd_bytes, size_t cont, FILE *arq);
```

onde:

- **`mem`**: representa a variável que armazena o conteúdo a ser gravado no arquivo
- **`qtd_bytes`**: representa o total em bytes que será escrito no arquivo
- **`cont`**: representa o número de blocos de tamanho *qtd_bytes* que serão escritos no arquivo
- **`arq`**: é a referência para o arquivo onde as informações serão escritas

fread

A função **fread()** pode ler qualquer tipo de dados e não apenas caracteres ou cadeia de caracteres. A sintaxe é:

```
int fread(void *mem, size_t qtd_bytes, size_t cont, FILE *arq);
```

onde:

- **mem:** representa a variável que vai receber conteúdo lido do arquivo
- **qtd_bytes:** representa o tamanho do bloco, em bytes, que será lido do arquivo
- **cont:** representa o número de blocos de tamanho *qtd_bytes* que serão lidos
- **arq:** é a referência para o arquivo que será lido

Exemplo 1: Problema Salvar Vetor

Suponha que você tenha um vetor de inteiros de 10 elementos, e gostaria de salvar em um arquivo esse vetor para posteriormente carregar do arquivo.

Exemplo 1: Solução c/ Arquivo Texto (1)

```
1. #include <stdio.h>
2.
3. int main() {
4.     int v[] = {5, 3, 2, 1, 4, 6, 7, 8, 9}, i;
5.     FILE *f;
6.
7.     f = fopen("vetor.txt", "w");
8.
9.     // se arquivo nao foi aberto corretamente
10.    if (f == NULL) {
11.        // imprima mensagem de erro e finalize
12.        perror("Erro ao abrir arquivo!\n");
13.        exit(1);
14.    }
15.
16.    // para cada elemento do vetor
17.    for (i = 0; i < 9; i++) {
18.        // escreva elemento no arquivo
19.        fprintf(f, "%d ", v[i]);
20.    }
21.
22.    fclose(f);
23.    return 0;
24. }
```

Exemplo 1: Solução c/ Arquivo Texto (2)



A screenshot of a text editor window titled "vetor.txt — Downloads". The window has a dark background and a light gray sidebar on the left. The main editing area shows a single line of text containing the numbers "5 3 2 1 4 6 7 8 9". The line number "1" is visible in the sidebar. The status bar at the bottom indicates "Line: 1:18", "Plain Text", and "Soft Tabs: 2".

```
1 5 3 2 1 4 6 7 8 9
```

Exemplo 1: Solução c/ Arquivo Texto (3)

```
1. #include <stdio.h>
2.
3. int main() {
4.     int v[9], i, n = 0, n_lido;
5.     FILE *f;
6.
7.     f = fopen("vetor.txt", "r");
8.     if (f == NULL) {
9.         perror("Erro ao abrir arquivo!\n");
10.        exit(1);
11.    }
12.
13.    // carregue elementos do arquivo para o vetor
14.    while(1) {
15.        n_lido = fscanf(f, "%d", &v[n]);
16.        if (n_lido == EOF)
17.            break;
18.        n = n + 1;
19.    }
```

```

1. #include <stdio.h>
2.
3. int main() {
4.     int v[9], i, n = 0, n_lido;
5.     FILE *f;
6.
7.     f = fopen("vetor.txt", "r");
8.     if (f == NULL) {
9.         perror("Erro ao abrir arquivo!\n");
10.        exit(1);
11.    }
12.
13.    // carregue elementos do arquivo para o vetor
14.    while(1) {
15.        n_lido = fscanf(f, "%d", &v[n]);
16.        if (n_lido == EOF)
17.            break;
18.        n = n + 1;
19.    }
20.
21.    for (i = 0; i < n; i++) {
22.        printf("%d ", v[i]);
23.    }
24.
25.    fclose(f);
26.    return 0;
27.}

```

%
5 3 2 1 4 6 7 8 9

Exemplo 1: Solução c/ Arquivo Binário (1)

```
1. #include <stdio.h>
2.
3. int main() {
4.     int v[100] = {5, 3, 2, 1, 4, 6, 7, 8, 9}, i;
5.     FILE *f;
6.
7.     // abra o arquivo
8.     f = fopen("vetor.bin", "wb");
9.
10.    // se arquivo nao foi aberto corretamente
11.    if (f == NULL) {
12.        perror("Erro ao abrir arquivo!\n");
13.        return 1;
14.    }
15.
16.    // escreva bytes no arquivo
17.    fwrite(v, sizeof(int), 9, f);
18.
19.    // feche o arquivo
20.    fclose(f);
21.    return 0;
22.}
```

Exemplo 1: Solução c/ Arquivo Binário (2)

```
1. #include <stdio.h>
2.
3. int main() {
4.     int v[100], i;
5.     FILE *f;
6.
7.     f = fopen("vetor.bin", "rb");
8.     if (f == NULL) {
9.         perror("Erro ao abrir arquivo!");
10.        return 1;
11.    }
12.
13.    // carregue bytes do arquivo para o vetor
14.    fread(v, sizeof(int), 9, f);
15.
16.    // imprima vetor
17.    for (i = 0; i < 9; i++) {
18.        // imprima elemento na tela
19.        printf("%d ", v[i]);
20.    }
21.
22.    fclose(f);
23.    return 0;
24.}
```

5	3	2	1	4	6	7	8	9
---	---	---	---	---	---	---	---	---

Exemplo 2: Crie Arquivo Binário

```
1. #include <stdio.h>
2.
3. struct clientData {
4.     int acctNum;
5.     char lastName[15];
6.     char firstName[10];
7.     double balance;
8. };
9.
10. int main () {
11.     int i;
12.     struct clientData blankClient = {0, "", "", 0.0};
13.     FILE *f;
14.
15.     // abra o arquivo no modo escrita binario
16.     f = fopen("credit.dat", "wb");
17.     // se arquivo nao foi aberto corretamente
18.     if (f == NULL) {
19.         // imprima mensagem de erro e finalize o programa
20.         printf("Erro ao abrir o arquivo\n");
21.         return 1;
22.     }
23.
24.     // escreva o arquivo
25.     for (i = 0; i < 10; i++) {
26.         struct clientData client;
27.         client.acctNum = i * 1000000;
28.         client.lastName = "Doe";
29.         client.firstName = "John";
30.         client.balance = 123456.78;
31.         fwrite(&client, sizeof(struct clientData), 1, f);
32.     }
33.     fclose(f);
34.     return 0;
35. }
```

```
1.#include <stdio.h>
2.
3.struct clientData {
4.    int acctNum;
5.    char lastName[15];
6.    char firstName[10];
7.    double balance;
8.};
9.
10.int main () {
11.    int i;
12.    struct clientData blankClient = {0, "", "", 0.0};
13.    FILE *f;
14.
15.    // abra o arquivo no modo escrita binario
16.    f = fopen("credit.dat", "wb");
17.    // se arquivo nao foi aberto corretamente
18.    if (f == NULL) {
19.        // imprima mensagem de erro e finalize o programa
20.        perror("Arquivo nao pode ser aberto!\n");
21.        return 1;
22.    }
23.    // repita 100 vezes
24.    for(i = 1; i <= 100; i++)
25.        // escreva no arquivo os bytes de um registro com os valores default
26.        fwrite(&blankClient, sizeof(struct clientData), 1, f);
27.    // feche o arquivo
28.    fclose(f);
29.    return 0;
30.}
```

Função fseek

fseek

<stdio.h>

```
int fseek ( FILE * stream, long int offset, int origin );
```

Reposition stream position indicator

Sets the position indicator associated with the *stream* to a new position.

For streams open in binary mode, the new position is defined by adding *offset* to a reference position specified by *origin*.

For streams open in text mode, *offset* shall either be zero or a value returned by a previous call to [ftell](#), and *origin* shall necessarily be `SEEK_SET`.

If the function is called with other values for these arguments, support depends on the particular system and library implementation (non-portable).

The *end-of-file internal indicator* of the *stream* is cleared after a successful call to this function, and all effects from previous calls to [ungetc](#) on this *stream* are dropped.

On streams open for update (read+write), a call to `fseek` allows to switch between reading and writing.

Link: <https://cplusplus.com/reference/cstdio/fseek/?kw=fseek>.

Função fseek (Cont.)

Constante	Posição de Referência
SEEK_SET	Início do arquivo
SEEK_CUR	Posição atual do ponteiro do arquivo
SEEK_END	Final do arquivo*

Constantes definidas em <stdio.h> para exclusivamente serem utilizadas como argumento para função fseek.

* As implementações da biblioteca podem não oferecer suporte significativo ao SEEK_END (portanto, o código que o utiliza não tem portabilidade padrão real).

Exemplo 3: Escreva no Arquivo Binário

```
1. ...
2. int main () {
3.     struct clientData client;
4.     FILE *f;
5.
6.     f = fopen("credit.dat", "rb+");
7.     if (f == NULL) {
8.         perror("Arquivo nao pode ser aberto!");
9.         return 1;
10.    }
11.    printf("Enter account number (1 to 100, 0 to end input)\n?");
12.    scanf("%d", &client.accNum);
13.    while (client.acctNum != 0) {
14.        printf("Enter lastname, firstname, balance\n?");
15.        scanf("%s%s%f", client.lastName, client.firstName, &client.balance);
16.        fseek(f, (client.accNum - 1) * sizeof(struct clientData), SEEK_SET);
17.        fwrite(&client, sizeof(struct clientData), 1, f);
18.        printf("Enter account number\n? ");
19.        scanf("%d", &client.acctNum);
20.    }
21.    fclose(f);
22.    return 0;
23.}
```

Exemplo 3: Escreva no Arquivo Binário (Cont.)

```
%  
Enter account number (1 to 100, 0 to end input)  
? 37  
Enter lastname, firstname, balance  
? Barker Doug 0.00  
Enter account number  
? 29  
Enter lastname, firstname, balance  
? Brown Nancy -24.54  
Enter account number  
? 96  
Enter lastname, firstname, balance  
? Stone Sam 34.98  
Enter lastname, firstname, balance  
? 0
```

Exemplo 4: Leia do Arquivo Binário

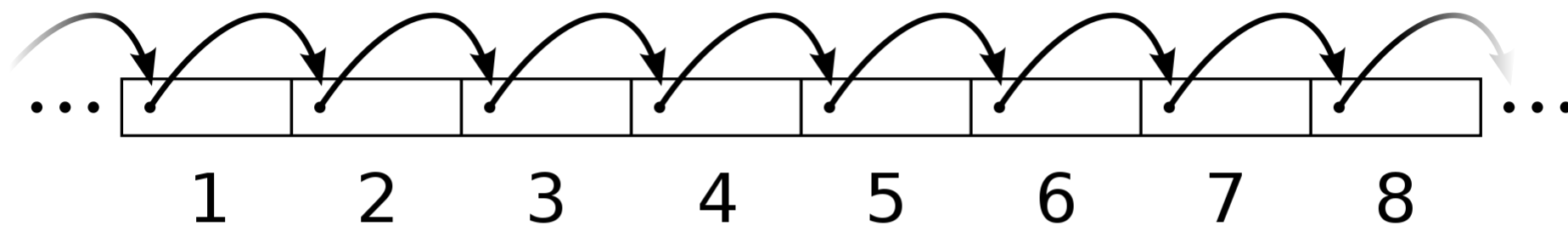
```
1. ...
2. int main () {
3.     int i, nread;
4.     struct clientData client;
5.     FILE *f;
6.
7.     f = fopen("credit.dat", "rb");
8.     if (f == NULL) {
9.         printf("Arquivo nao pode ser aberto!\n");
10.        return 1;
11.    }
12.    printf("%-6s%-16s%-11s%10s\n", "Acct", "Last Name", "First Name", "Balance");
13.    while (1) {
14.        nread = fread(&client, sizeof(struct clientData), 1, f);
15.        if (nread <= 0)
16.            break;
17.        if (client.accNum != 0) {
18.            printf("%-6d%-16s%-11s%10.2f\n",
19.                client.accNum, client.lastName,
20.                client.firstName, client.balance);
21.        }
22.    }
23.    fclose(f);
24.    return 0;
25.}
```

Exemplo 4: Leia do Arquivo Binário (Cont.)

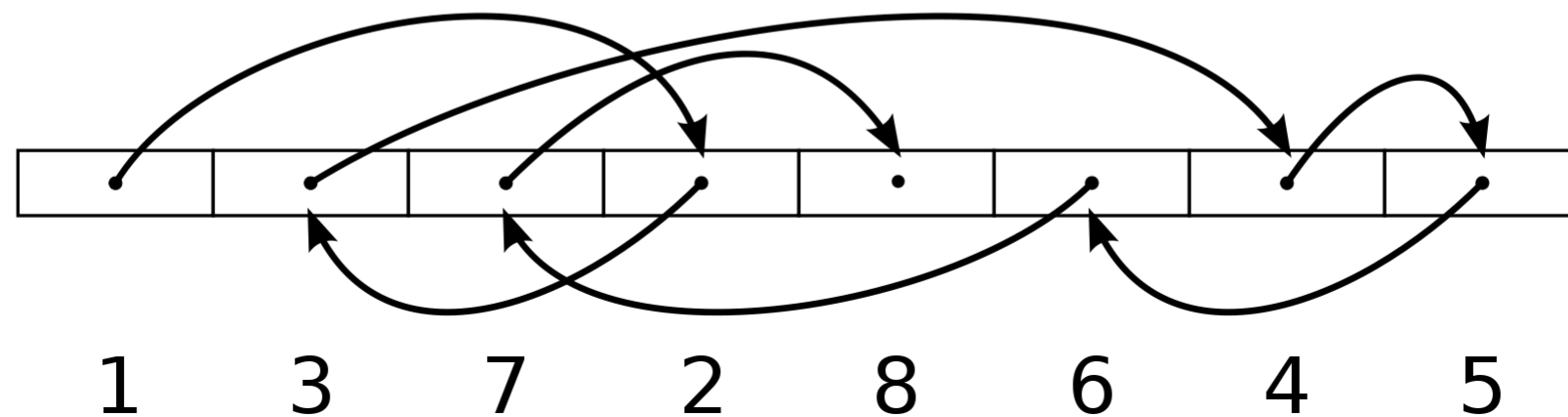
Acct	Last Name	First Name	Balance
29	Brown	Nancy	-24.54
33	Dunn	Stacey	314.33
37	Barker	Doug	0.00
88	Smith	Dave	258.34
96	Stone	Sam	34.98

Acesso Sequencial vs Acesso Aleatório

Sequential access (Arquivos Texto)



Random access (Arquivos Binário)



Referências

- ASCENCIO, A. F. G.; CAMPOS, E. A. V. Fundamentos da programação de computadores: algoritmos, Pascal, C/C++ e Java. 3. ed. São Paulo: Pearson, 2012.
- DEITEL, P.; DEITEL, H. C: Como Programar. 6^a ed. São Paulo: Pearson, 2011.
- PIVA, D.J. et al. Algoritmos e programação de computadores. Rio de Janeiro: Elsevier, 2012.